

AuditPlatform

Rapport d'analyse — tunisia-real-estate

Généré le 27/03/2026 à 22:14

■ Informations du projet

Nom du projet	tunisia-real-estate
URL du projet	yosrchiha01/tunisia-real-estate
Branche analysée	main
Date de l'analyse	27/03/2026 21:36
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	77	77/100
Sécurité	30	30/100
Performance	35	35/100

■ ■ Vulnérabilités (76)

A01 — BROKEN ACCESS CONTROL
Sévérité : HAUTE
Fichier : frontend/src/pages/dashboard/AddPropertyForm.tsx — ligne 34
Correction : Vérifiez que l'endpoint /api/properties est protégé par authentification et que les requêtes POST sont validées correctement.
A01 — BROKEN ACCESS CONTROL
Sévérité : HAUTE

Fichier : frontend/src/pages/dashboard/buyer.tsx — ligne 24

Correction : Vérifiez que l'endpoint /api/properties est protégé par authentification et que les requêtes GET sont validées correctement.

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : frontend/src/services/api.ts — ligne 14

Correction : Utilisez une méthode de chiffrement plus sécurisée que le token Bearer pour les requêtes API.

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : frontend/src/services/authService.ts — ligne 34

Correction : Vérifiez que les tokens JWT sont générés avec une clé secrète sécurisée et que l'algorithme de signature est correct.

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/src/services/propertyService.ts — ligne 6

Correction : Utiliser des requêtes préparées ou des transactions pour améliorer les performances

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/prisma/seed.ts — ligne 0

Correction : Considérer la séparation de la logique de création de données et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/config/cors.ts — ligne 0

Correction : Considérer l'utilisation d'une bibliothèque de configuration pour gérer les paramètres de CORS

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/index.ts — ligne 0

Correction : Considérer la séparation de la logique de configuration et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/middleware/auth.ts — ligne 0

Correction : Considérer l'utilisation d'une bibliothèque de middleware pour gérer les authentications

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/routes/auth.ts — ligne 0

Correction : Considérer la séparation de la logique de routes et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/routes/properties.ts — ligne 0

Correction : Considérer la séparation de la logique de routes et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/services/authService.ts — ligne 0

Correction : Considérer la séparation de la logique de services et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : backend/src/services/propertyService.ts — ligne 0

Correction : Considérer la séparation de la logique de services et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/App.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/components/ErrorBoundary.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/components/ProtectedRoute.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/contexts/AuthContext.tsx — ligne 0

Correction : Considérer la séparation de la logique de contextes et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/contexts/useAuth.ts — ligne 0

Correction : Considérer la séparation de la logique de contextes et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/pages/Accueil.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/pages/Dashboard.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/pages/Home.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/pages/Login.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/src/pages/Register.tsx — ligne 0

Correction : Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/index.ts — ligne 1

Correction : Ajouter une docstring pour expliquer le but de l'application

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/prisma/seed.ts — ligne 15

Correction : Ajouter des commentaires pour expliquer les étapes de la mise à jour de la base de données

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/src/middleware/auth.ts — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `authMiddleware`

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/src/routes/auth.ts — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `auth`

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/src/services/authService.ts — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `registerUser`

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/src/services/propertyService.ts — ligne 1

Correction : Ajouter des commentaires pour expliquer les étapes de la récupération des propriétés

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/App.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `App`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/components/ErrorBoundary.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `ErrorBoundary`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/components/ProtectedRoute.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `ProtectedRoute`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/contexts/AuthContext.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `AuthProvider`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/contexts/useAuth.ts — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `useAuth`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/pages/Accueil.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `Accueil`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/pages/Dashboard.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `Dashboard`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/pages/Home.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `Home`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/pages/Login.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `Login`

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/pages/Register.tsx — ligne 1

Correction : Ajouter une docstring pour expliquer le but de la fonction `Register`

SOLID-S

Sévérité : MOYENNE

Fichier : backend/src/services/authService.ts — ligne 24

Correction : La méthode registerUser peut être divisée en plusieurs fonctions plus petites et plus faciles à tester. Par exemple, vous pouvez créer une fonction pour vérifier si l'utilisateur existe déjà, une autre pour hasher le mot de passe, etc.

SOLID-S

Sévérité : MOYENNE

Fichier : backend/src/services/authService.ts — ligne 64

Correction : La méthode loginUser peut être divisée en plusieurs fonctions plus petites et plus faciles à tester. Par exemple, vous pouvez créer une fonction pour vérifier si l'utilisateur existe, une autre pour vérifier le mot de passe, etc.

Testabilité

Sévérité : MOYENNE

Fichier : backend/src/test-auth-simple-debug.ts — ligne 24

Correction : La fonction testAuth utilise des requêtes HTTP pour tester les routes. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : backend/src/test-auth-simple.ts — ligne 24

Correction : La fonction testAuth utilise des requêtes HTTP pour tester les routes. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : backend/src/test-deps.ts — ligne 24

Correction : La fonction testAuth utilise des requêtes HTTP pour tester les routes. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : backend/src/test-service-direct.ts — ligne 24

Correction : La fonction testServiceDirectly utilise des requêtes HTTP pour tester les services. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/src/components/ErrorBoundary.tsx — ligne 24

Correction : La fonction componentDidCatch ne gère pas les erreurs de manière efficace. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/src/components/ProtectedRoute.tsx — ligne 24

Correction : La fonction ProtectedRoute utilise des requêtes HTTP pour tester les routes. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/src/contexts/AuthContext.tsx — ligne 24

Correction : La fonction fetchUserProfile utilise des requêtes HTTP pour tester les services. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

Testabilité

Sévérité : MOYENNE

Fichier : frontend/src/contexts/useAuth.ts — ligne 24

Correction : La fonction useAuth utilise des requêtes HTTP pour tester les services. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles.

A08 — DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : frontend/src/services/propertyService.ts — ligne 10

Correction : Vérifiez que les données récupérées de l'API sont validées correctement avant d'être utilisées.

SELECT*

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/AddPropertyForm.tsx — ligne 34

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Requêtes exécutées à l'intérieur de boucles

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/AddPropertyForm.tsx — ligne 34

Correction : Utiliser des requêtes préparées ou des transactions pour améliorer les performances

Requêtes exécutées à l'intérieur de boucles

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/buyer.tsx — ligne 24

Correction : Utiliser des requêtes préparées ou des transactions pour améliorer les performances

SELECT*

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/buyer.tsx — ligne 24

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Requêtes exécutées à l'intérieur de boucles

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/homeowner.tsx — ligne 14

Correction : Utiliser des requêtes préparées ou des transactions pour améliorer les performances

SELECT*

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/homeowner.tsx — ligne 14

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Requêtes exécutées à l'intérieur de boucles

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/renter.tsx — ligne 24

Correction : Utiliser des requêtes préparées ou des transactions pour améliorer les performances

SELECT*

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/renter.tsx — ligne 24

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

SELECT*

Sévérité : MOYENNE

Fichier : frontend/src/services/propertyService.ts — ligne 6

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/AddPropertyForm.tsx — ligne 14

Correction : Ajouter une docstring pour expliquer le rôle de la fonction handleSubmit

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/AddPropertyForm.tsx — ligne 30

Correction : Ajouter un commentaire pour expliquer le fonctionnement de la fonction handleSubmit

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/src/services/api.ts — ligne 10

Correction : Ajouter un commentaire pour expliquer le fonctionnement de l'intercepteur de requête

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/src/services/authService.ts — ligne 20

Correction : Ajouter un commentaire pour expliquer le fonctionnement de la fonction register

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/AddPropertyForm.tsx — ligne 34

Correction : La fonction handleSubmit est trop longue et fait plusieurs choses. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à tester.

Architecture

Sévérité : MOYENNE

Fichier : frontend/src/pages/dashboard/homeowner.tsx — ligne 14

Correction : Il est préférable de séparer la logique de présentation de la logique métier pour améliorer la séparation des couches et la testabilité.

Gestion erreurs

Sévérité : MOYENNE

Fichier : frontend/src/services/api.ts — ligne 14

Correction : Il est préférable de gérer les erreurs de manière plus spécifique plutôt que de simplement afficher un message d'erreur générique.

Lisibilité

Sévérité : FAIBLE

Fichier : backend/index.ts — ligne 0

Correction : Ajouter une description pour le fichier index.ts

Nommage ambigu

Sévérité : FAIBLE

Fichier : backend/src/config/cors.ts — ligne 1

Correction : Considérer un nom plus explicite pour la variable `corsConfig`

Gestion erreurs

Sévérité : FAIBLE

Fichier : backend/src/routes/auth.ts — ligne 34

Correction : Les erreurs sont gérées de manière globale dans le fichier auth.ts. Il est préférable de gérer les erreurs directement dans les fonctions qui les génèrent.

Gestion erreurs

Sévérité : FAIBLE

Fichier : backend/src/routes/auth.ts — ligne 64

Correction : Les erreurs sont gérées de manière globale dans le fichier auth.ts. Il est préférable de gérer les erreurs directement dans les fonctions qui les génèrent.

Gestion erreurs

Sévérité : FAIBLE

Fichier : backend/src/routes/properties.ts — ligne 14

Correction : La fonction createProperty ne gère pas les erreurs. Il est préférable de gérer les erreurs directement dans la fonction.

A05 — SECURITY MISCONFIGURATION

Sévérité : FAIBLE

Fichier : frontend/tailwind.config.js — ligne 0

Correction : Vérifiez que les fichiers de configuration ne contiennent pas de secrets ou de données sensibles.

A05 — SECURITY MISCONFIGURATION

Sévérité : FAIBLE

Fichier : frontend/vite.config.ts — ligne 0

Correction : Vérifiez que les fichiers de configuration ne contiennent pas de secrets ou de données sensibles.

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/src/pages/dashboard/buyer.tsx — ligne 24

Correction : Renommer la variable 'filtered' en quelque chose de plus explicite

Gestion erreurs

Sévérité : FAIBLE

Fichier : frontend/src/pages/dashboard/buyer.tsx — ligne 24

Correction : Il est préférable de gérer les erreurs de manière plus spécifique plutôt que de simplement afficher un message d'erreur générique.

Gestion erreurs

Sévérité : FAIBLE

Fichier : frontend/src/services/propertyService.ts — ligne 6

Correction : Il est préférable de gérer les erreurs de manière plus spécifique plutôt que de simplement afficher un message d'erreur générique.

■ Recommandations

1. Séparer la logique de configuration et de la logique métier

Considérer la séparation de la logique de configuration et de la logique métier dans des fichiers séparés pour améliorer la maintenabilité et la lisibilité du code.

2. Utiliser des bibliothèques de configuration pour gérer les paramètres de CORS

Considérer l'utilisation d'une bibliothèque de configuration pour gérer les paramètres de CORS pour améliorer la sécurité et la flexibilité du code.

3. Séparer la logique de services et de la logique métier

Considérer la séparation de la logique de services et de la logique métier dans des fichiers séparés pour améliorer la maintenabilité et la lisibilité du code.

4. Séparer la logique de composants et de la logique métier

Considérer la séparation de la logique de composants et de la logique métier dans des fichiers séparés pour améliorer la maintenabilité et la lisibilité du code.

5. Séparer la logique de contextes et de la logique métier

Considérer la séparation de la logique de contextes et de la logique métier dans des fichiers séparés pour améliorer la maintenabilité et la lisibilité du code.

6. Ajouter des commentaires pour expliquer les étapes de la mise à jour de la base de données

Ajouter des commentaires pour expliquer les étapes de la mise à jour de la base de données dans le fichier ``backend/prisma/seed.ts``

7. Considérer un nom plus explicite pour la variable ``corsConfig``

Considérer un nom plus explicite pour la variable ``corsConfig`` dans le fichier ``backend/src/config/cors.ts``

8. Ajouter des commentaires pour expliquer les étapes de la récupération des propriétés

Ajouter des commentaires pour expliquer les étapes de la récupération des propriétés dans le fichier ``backend/src/services/propertyService.ts``

9. Diviser les fonctions en plus petites et plus faciles à tester

Les fonctions telles que `registerUser` et `loginUser` peuvent être divisées en plusieurs fonctions plus petites et plus faciles à tester. Par exemple, vous pouvez créer une fonction pour vérifier si l'utilisateur existe déjà, une autre pour hasher le mot de passe, etc.

10. Gérer les erreurs directement dans les fonctions qui les génèrent

Les erreurs sont gérées de manière globale dans les fichiers `auth.ts` et `properties.ts`. Il est préférable de gérer les erreurs directement dans les fonctions qui les génèrent.

11. Utiliser des tests unitaires pour tester les fonctions individuelles

Les tests unitaires peuvent être utilisés pour tester les fonctions individuelles de manière efficace. Il est préférable d'utiliser des tests unitaires pour tester les fonctions individuelles plutôt que de tester les routes ou les services en entier.

12. Utilisez une authentification et une autorisation robustes

Vérifiez que les endpoints API sont protégés par authentification et que les requêtes sont validées correctement. Utilisez une méthode de chiffrement sécurisée pour les tokens JWT.

13. Vérifiez les données récupérées de l'API

Vérifiez que les données récupérées de l'API sont validées correctement avant d'être utilisées. Utilisez des méthodes de validation robustes pour éviter les injections de code.

14. Utilisez des méthodes de chiffrement sécurisées

Utilisez des méthodes de chiffrement sécurisées pour les données sensibles, telles que les mots de passe et les clés API. Évitez d'utiliser des méthodes de chiffrement dépréciées ou non sécurisées.

15. Utiliser des requêtes préparées ou des transactions

Les requêtes préparées ou les transactions peuvent améliorer les performances en réduisant le nombre de requêtes exécutées à l'intérieur de boucles. Exemple de code optimisé :

16. Utiliser les colonnes nécessaires

Utiliser les colonnes nécessaires au lieu de `SELECT *` peut améliorer les performances en réduisant la quantité de données transférées. Exemple de code optimisé :

17. Ajouter des docstrings pour les fonctions

Ajouter des docstrings pour expliquer le rôle et le fonctionnement de chaque fonction

18. Utiliser des commentaires pour expliquer les algorithmes complexes

Utiliser des commentaires pour expliquer les algorithmes complexes et les fonctionnalités spécifiques

19. Renommer les variables pour améliorer la lisibilité

Renommer les variables pour améliorer la lisibilité et la compréhension du code

20. Séparer la logique de présentation de la logique métier

Il est préférable de séparer la logique de présentation de la logique métier pour améliorer la séparation des couches et la testabilité. Cela peut être fait en créant des composants de présentation séparés des composants de métier.

21. Utiliser des exceptions spécifiques

Il est préférable d'utiliser des exceptions spécifiques plutôt que des exceptions génériques pour améliorer la compréhension et la gestion des erreurs.

22. Améliorer la testabilité

Il est préférable d'améliorer la testabilité en utilisant des techniques telles que le mocking et le stubbing pour faciliter les tests unitaires et intégrés.